

Off-Screen — A Healthy Alternative Activity Generator

Derya Demirkol · Group 1 · github.com/derya-demirkol/digital-addiction-hackathon

1. Problem and Target User

Many people don't reach for their phone because they specifically want social media, gaming, or video — they reach for it because they're bored, tired, avoiding a task, or looking for a small hit of stimulation, and the phone is simply the fastest thing available. Telling someone to "just use your phone less" doesn't remove that underlying need; it only removes the tool they were using to meet it, which is why willpower-only approaches tend to fail quickly.

Off-Screen targets that gap directly. The user is a student or young professional who already knows which habit they want to cut back on (for example, scrolling social media before bed) but struggles to think of a good alternative in the moment. Rather than lecturing the user about screen time, the tool asks what they're trying to replace, how much time and what setting they have, and what they're generally into — and returns three concrete, doable, off-screen activities sized to fit.

2. Solution Overview and Model Used

The prototype is a single-page web application. The user fills in four fields — the habit they want to reduce, time available, physical context (home, outdoors, commuting), and personal interests — and the app sends this to Claude (claude-sonnet-4-6) through the Anthropic Messages API. The model returns three tailored activity suggestions as structured JSON, which the interface renders as a set of short, readable cards. The full loop (input → generation → rendered output) runs end to end with real API calls, using the example described in Section 4.

3. Key Prompts and Design Rationale

The core of the tool is a single system prompt that combines two techniques covered in the module: structured step-by-step instruction and structured (JSON) output. An early zero-shot version of the prompt ("suggest 3 offline activities based on this input") produced generic, repetitive answers — mostly "go for a walk" or "read a book" regardless of what the user typed. Adding an explicit reasoning sequence measurably improved relevance, because it forces the model to first identify the underlying need behind the habit before matching it to an activity, rather than pattern-matching directly from habit to a generic suggestion.

System prompt (abridged):

You are a digital well-being assistant. Your job is to suggest OFF-SCREEN activities that fit the user's interests, available time, and setting.

Think step by step (but return only the JSON result):

1. Infer the likely need behind the habit (boredom, escape, social connection, stimulation, etc.)
2. Choose 3 different activities that realistically fit the given time and setting and could meet that need off-screen
3. Write a short, non-judgmental, encouraging description for each (max 2 sentences)
4. Do NOT use diagnostic language or call the user's behavior an "addiction"

Return valid JSON only, matching this schema:

```
{ "activities": [ { "title", "description", "duration_fit" } ] }
```

I chose structured output (strict JSON, single schema, no free text) over an open-ended chat-style response for two reasons: it makes the output reliable enough to render in the UI without extra parsing logic, and it keeps the model's answer scoped to exactly what was asked, which reduces the chance of it drifting into commentary about the user's habit. I deliberately avoided few-shot examples in the final version: early tests showed the model anchoring too closely to the example activities I provided (e.g. always suggesting "journaling" because it appeared in a sample), so the step-by-step instruction alone gave more varied, better-targeted results than few-shot did for this particular task.

4. Working Example

Input — habit: "scrolling social media before bed"; time available: 30 minutes; context: at home; interests: music, drawing, spending time with friends.

Output — three suggestions were returned, including a low-effort creative option (sketching to a short playlist instead of scrolling), a connection-oriented option (a quick voice or video call with a friend instead of passively watching others' posts), and a wind-down option matched to the bedtime context. Each came with a short rationale and a note on why it fit the 30-minute window. Full input/output examples and screenshots are included with the demo video.

5. Challenges

- Team continuity: I started this project as part of a five-person team (Group 1), but my teammates stopped communicating partway through the build weekend. I scoped the project down and completed it solo, which limited the amount of testing and iteration I was able to do in the remaining time.
- Prompt reliability: even with explicit JSON-only instructions, the model occasionally still returns output that fails to parse as valid JSON — this is an intermittent issue I was not able to fully eliminate within the weekend. I mitigated it with an automatic retry (up to three attempts, each with a stronger reminder) and a graceful fallback that displays the model's raw text instead of a dead-end error if parsing still fails, so the tool never breaks outright during a demo.
- Working solo on a tight timeline: since I completed this project alone rather than with the original five-person team, I had limited time to test the prototype across a wide range of inputs. The retry/fallback approach above is a practical mitigation given that constraint, but a version tested with more time and more reviewers would likely catch and fix more edge cases.
- Avoiding generic output: as noted in Section 3, the first prompt draft produced repetitive, one-size-fits-all suggestions; iterating toward the step-by-step version was necessary to get answers that actually responded to the user's specific input.

6. Ethical Evaluation

Overclaiming / medical framing: the tool could easily read as diagnosing the user with a form of "addiction." I mitigated this by explicitly instructing the model never to use diagnostic language, and by adding a visible disclaimer on the page stating the tool is not medical advice and does not diagnose anything.

Judgmental tone: because the input is emotionally loaded (users may already feel bad about their habit), a scolding tone would discourage use. The system prompt requires a "non-judgmental, encouraging" register, and outputs were manually reviewed during testing to confirm none of the generated text sounded critical or shaming.

Privacy and sensitive data: the only data sent to the API is the four short fields the user chooses to type — no account, no persistent storage, and no logging beyond what is needed to render the response in the browser session. Users are not asked for any identifying or health-related information, and none is required for the tool to work.

Vulnerable users and bias: because the model generates open-ended suggestions, there is a small risk of activities that assume a certain budget, mobility level, or social situation (e.g. suggesting a group activity to someone who is isolated). This is a known limitation I did not fully solve in the prototype; a future iteration could ask an explicit accessibility/budget question and pass it into the prompt so suggestions are filtered accordingly.

Unsafe or misleading suggestions: since suggestions are generated per request rather than pulled from a vetted list, there is a low but non-zero chance the model proposes something impractical or, in an edge case, unsafe for a given context. For a production version, I would add a lightweight content check before displaying suggestions to the user.